

CH12 现代运维体系与云原生技术图谱

考试复习笔记

来源：开源操作系统课件（28 页） 日期：2026 年 6 月 23 日

1 现代运维体系 [p.9–18]

1.1 现代生产系统的核心问题 [p.9]

- 新版本如何安全发布？
- 系统是否正在正常运行？
- 故障发生后如何快速定位？
- 数据丢失后能否恢复？
- 安全问题如何提前发现？
- 基础设施如何避免手工漂移？

1.2 CI/CD 自动化交付 [p.10–15]

1.2.1 传统发布 vs CI/CD [p.10]

传统：本地打包 → 手工上传 → 手工改配置 → 手工停旧 → 手工启新 → 手工回滚。步骤依赖个人经验、缺少测试门禁、难以追踪。

1.2.2 CI/CD 定义 [p.11]

概念	说明
CI (Continuous Integration)	代码提交后自动拉取 → 自动编译 → 自动测试 → 自动检查代码质量和依赖风险 → 尽早发现集成问题
CD (Delivery/Deployment)	自动构建制品 → 推送镜像仓库 → 部署到测试环境 → 审批后部署到生产 → 支持观察和快速回滚

1.2.3 CI/CD 重要概念 [p.13–14]

- 代码仓库：保存代码、构建脚本、部署配置、基础设施声明文件和变更历史
- 制品：jar/war/静态文件/容器镜像。容器化时代，镜像通常成为核心交付制品
- 镜像仓库：保存和分发镜像。生产环境应使用明确版本，不长期依赖 latest

- **流水线**：把构建、测试、扫描、部署和回滚变成可重复流程

1.2.4 发布策略 [p.13]

策略	思想	优点	风险
停机发布	停止旧版本，启动新版本	简单	有中断
滚动发布	逐步替换实例	不必整体停机	新旧版本短暂共存
蓝绿发布	保留蓝绿两套环境，切换流量	回滚快	资源成本高
金丝雀发布	少量流量先进入新版本	风险可控	需要流量控制和观测
灰度发布	按用户/地区/比例逐步放量	精细控制	策略复杂

1.2.5 CI/CD 核心纪律 [p.15]

1. 每次发布都由同一套流程完成
2. 每次发布都有版本、日志和记录
3. 每次发布前都有测试和检查
4. 每次发布后都能观察状态
5. 每次失败都能回滚和复盘

1.3 可观测性 [p.16–18]

1.3.1 三大支柱 [p.16]

支柱	回答的问题
日志 (Logs)	” 发生了什么 ”——事件记录
指标 (Metrics)	” 系统状态如何变化 ”——数值趋势
链路追踪 (Traces)	” 一个请求经过了哪些服务 ”——调用链路

核心目的：不是为了做漂亮大屏，而是为了形成**证据链**。

1.3.2 可观测性排查范例 [p.17]

以 PetStore 下单接口变慢为例：

- **指标**：P99 延迟从 300ms → 5s，数据库连接池等待时间升高
- **日志**：应用日志出现大量 SQL timeout，MySQL 慢查询日志出现订单查询慢 SQL
- **链路追踪**：请求大部分耗时发生在 order-service → MySQL
- **结论**：问题更可能在数据库查询/索引/连接池，而不是 Nginx

2 云原生架构 [p.20–23]

2.1 云原生定义 [p.20]

云原生 (Cloud Native) 是一组利用云计算交付模型的优势来构建和运行可伸缩应用的方法，包含容器、服务网格、微服务、不可变基础设施和声明式 API。

核心理念：

- 不可变基础设施：服务器配置好后不再修改，需要变更就替换
- 声明式配置：描述期望状态，系统自动维持
- 自服务：开发者可以自助部署和运维
- 弹性伸缩：根据负载自动调整资源
- 故障设计：假设一切都会故障，设计自动恢复机制

2.2 云原生架构优势 [p.23]

1. 弹性：根据负载扩容缩容
2. 自动化：构建/测试/发布/回滚/扩缩容/部分故障恢复由平台自动执行
3. 标准化：镜像、声明式配置、Kubernetes API、GitOps
4. 可观测性：日志/指标/链路追踪/告警成为系统基础能力
5. 可靠性：多副本、健康检查、故障恢复、发布策略、SRE 方法
6. 团队协作：开发/测试/运维/安全通过流水线、配置仓库、监控平台协作

3 技术图谱与后续学习 [p.25–27]

3.1 面向开发者的学习路线 [p.26]

1. 强化 Linux 和网络基础：进程/线程、端口/TCP 连接、文件描述符、系统资源限制、Shell 自动化
2. 生产化 Web 应用开发：配置外置、健康检查接口、日志规范、错误处理、接口幂等性、数据库连接池、缓存使用模式、限流和降级
3. 容器化交付：Dockerfile 编写、镜像分层优化、安全基础镜像、环境变量和 Secret、本地 Compose
4. 可观测性意识：日志能否定位问题？指标能否反映系统状态？错误信息是否可理解？请求链路能否追踪？告警能否指向用户影响？

3.2 面向运维/SRE 的学习路线 [p.27]

1. Linux 系统深入：启动、systemd、内核参数、文件系统、网络栈、性能分析、日志系统、安全机制
2. 自动化运维：Shell、Python、Ansible、批量任务、配置管理、巡检和自愈脚本
3. 监控与可观测性：Prometheus、Grafana、日志系统、链路追踪、告警规则、Dashboard 设计
4. Kubernetes 平台运维：集群安装升级、网络插件、存储插件、RBAC、资源配额、节点维护、应用发布、平台故障排查

4 考试题型 [p.28]

题型	分值	说明
单项选择	$2 \times 20 = 40$ 分	基础知识
名词解释	$3 \times 5 = 15$ 分	核心概念
脚本/配置阅读或填空	$3 \times 5 = 15$ 分	实操能力
简答题	$5 \times 6 = 30$ 分	综合分析